

Conception et Implémentation d'un Moteur de Recherche Sémantique

Basé sur PostgreSQL + pgvector

Idriss Yacine ZIADI, Rayan BOUKAKIOU

École nationale Supérieure d'Informatique (ESI) – Alger
2CS SIQ1 – Bases de Données Avancées – Année 2025–2026

Encadré par : Mme AMROUCHE Karima

Mini-projet BDA

Juin 2026

Plan

1. Problématique
2. Architecture & Méthodologie
3. Implémentation
4. Résultats
5. Conclusion
6. Références

En bref

- 1 964 documents (AG News)
- Embeddings 384 dimensions
- Precision@5 : **0.81**
- Temps de réponse : **19.8 ms**

Problématique

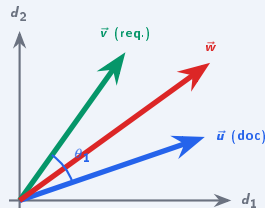
Recherche lexicale (TF-IDF)

- Correspondance **exacte** de mots-clés
- “automobile” $\neq$ “voiture”
- Ignore synonymie et paraphrase
- Rapide mais limitée en pertinence

Recherche sémantique

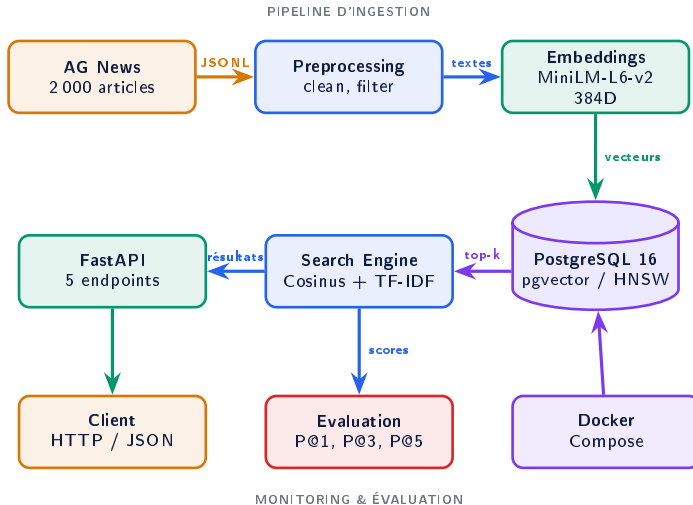
- Recherche par le **sens** du texte
- “automobile” $\approx$ “voiture” $\approx$ “véhicule”
- Capture les relations sémantiques
- Meilleure pertinence

Similarité cosinus

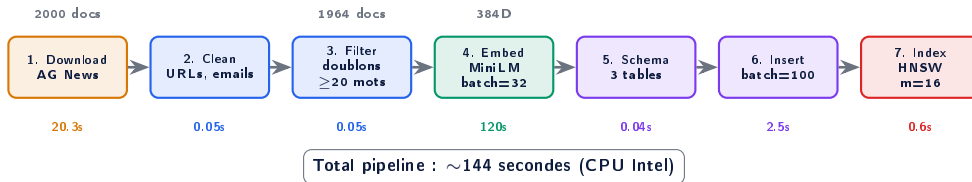


Question : *Comment implémenter une recherche sémantique performante avec des technologies open-source ?*

Architecture du système



Pipeline de traitement



Modèle d'embeddings

- **all-MiniLM-L6-v2**
(Sentence-Transformers)
- 384 dimensions, 22.7M paramètres
- Score STS Benchmark : 0.8282

Index HNSW

- $m = 16$ (connexions par nœud)
- $ef_construction = 64$
- Recall quasi-parfait, construction 0.6s

Implémentation

Schéma de la base (3 tables)

documents

id **SERIAL PK**
title **TEXT NOT NULL**
content **TEXT NOT NULL**
category **VARCHAR(100)**
embedding **vector(384)**
created_at **TIMESTAMP**

search_logs

id **SERIAL PK**
query_text **TEXT**
query_embedding **vector(384)**
search_type **VARCHAR(50)**
execution_time_ms **FLOAT**

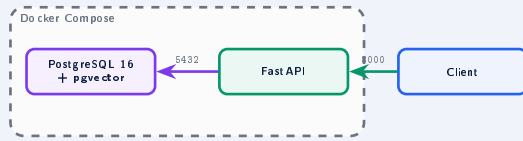
evaluation_results

id **SERIAL PK**
query_text **TEXT**
search_type **VARCHAR(50)**
precision_at_k **FLOAT**
k_value **INTEGER**

API REST (FastAPI)

Méthode	Endpoint	Description
GET	/health	État de santé
POST	/search/semantic	Recherche sémantique
POST	/search/lexical	Recherche TF-IDF
POST	/search/compare	Comparaison
GET	/stats	Statistiques

Infrastructure Docker



Résultats : Précision

Precision@k (20 requêtes, 1 964 documents)

Méthode	P@1	P@3	P@5
Sémantique	0.80	0.85	0.81
TF-IDF	0.80	0.70	0.76
Δ	0%	+21.4%	+6.6%

Observation clé : P@1 identique (0.80), mais l'avantage sémantique apparaît à $k = 3$ (**+21.4%**) grâce à la capture de synonymie et paraphrase.

Taux de victoire (20 requêtes)

Sém. : 8 (40%)

Égalité : 10 (50%)

2

TF-IDF : 10%

Par catégorie (P@5)

Cat.	Sem.	TF-IDF	Δ
Sci/Tech	0.96	0.92	+4.3%
Business	0.84	0.68	+23.5%
Sports	0.76	0.80	-5.0%
World	0.68	0.64	+6.3%

Résultats : Performance

Temps de réponse (20 requêtes)

Méthode	Moy.	Méd.	Max	P95
Sém.	19.8	18.5	32.1	26.2
TF-IDF	4.2	3.8	8.5	5.9

TF-IDF est $\sim 4.7\times$ plus rapide

Décomposition du temps sémantique

Encoding CPU : ~ 12 ms

HNSW : ~ 7 ms

Total = 19.8 ms

Analyse

- Le goulot d'étranglement est l'**encoding CPU** (~ 12 ms), pas la recherche vectorielle
- Avec GPU : temps attendu < 5 ms
- 19.8 ms reste **largement sous le seuil** perceptible (100 ms)
- HNSW offre un **recall quasi-parfait** avec seulement 7 ms de recherche

Trade-off : La recherche sémantique est $4.7\times$ plus lente mais **+6.6%** plus précise. Le surcoût de 15 ms est négligeable pour l'utilisateur.

Conclusion

Bilan

- Moteur de recherche sémantique **fonctionnel**
- Precision@5 : **0.81** vs 0.76 (TF-IDF)
- Temps de réponse : **19.8 ms** (< 100 ms)
- Architecture **reproductible** (Docker + Make)
- API REST complète (5 endpoints)
- pgvector s'intègre nativement dans PostgreSQL

Perspectives

- **GPU** pour réduire l'encoding (< 5 ms)
- **Recherche hybride** (sémantique + lexicale)
- **Corpus plus large** (> 100K documents)
- **Modèles multilingues** (français, arabe)
- **Re-ranking** avec cross-encoders

Apport principal : pgvector permet d'ajouter la recherche vectorielle à PostgreSQL **sans changer de SGBD** – un avantage décisif pour l'adoption en production.

Références

- [1] pgvector (2024). *Open-source vector similarity search for Postgres*.
github.com/pgvector/pgvector
- [2] Malkov, Y. A., Yashunin, D. A. (2020). *Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs*. IEEE TPAMI, 42(4), pp.824–836.
- [3] Reimers, N., Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
- [4] Zhang, X., Zhao, J., LeCun, Y. (2015). *Character-level Convolutional Networks for Text Classification*. NeurIPS 2015.

Merci !

Questions & Discussion

Mini-projet réalisé dans le cadre du cours de Bases de Données Avancées.

`github.com/idrissziadi/pgvector-semantic-search`